

- Description

This module manages a smart solar panel system, ensuring that:

1. **The power output of solar panels is continuously monitored to optimize energy generation.**
2. **If power generation drops below an efficiency threshold, the panel tilt angle is adjusted for better sunlight exposure.**
3. **If the power output reaches the optimal range, the tilt remains unchanged to avoid unnecessary movement.**
4. **A log of all power readings, tilt adjustments, and efficiency improvements is maintained for energy optimization.**

This system helps maximize solar energy utilization, improve efficiency, and reduce maintenance costs.

- Specification

module SmartSolarPanelControl

var currentPowerOutput, maxOptimalPower, minOptimalPower: real

var panelTiltAngle: real

process updatePowerOutput(changeAmount: real)

functional scenario form:

currentPowerOutput >= minOptimalPower && Power output maintained optimally.

||

currentPowerOutput < minOptimalPower && panelTiltAngle = panelTiltAngle + 5.0, Panel tilt increased to enhance sunlight absorption.

||

currentPowerOutput > maxOptimalPower && panelTiltAngle = panelTiltAngle - 5.0, Panel tilt reduced to prevent overexposure.

end process

end module

- Java Code

```
package code;
class SmartSolarPanel {
    private double maxOptimalPower;
    private double minOptimalPower;
    private double currentPowerOutput;
    private double panelTiltAngle;
    public SmartSolarPanel(double maxOptimalPower, double minOptimalPower, double initialPower, double initialAngle) {
        this.maxOptimalPower = maxOptimalPower;
        this.minOptimalPower = minOptimalPower;
        this.currentPowerOutput = initialPower;
        this.panelTiltAngle = initialAngle;
    }
}
```

```

    }
    public void updatePowerOutput(double changeAmount) {
        currentPowerOutput += changeAmount;
        adjustTiltAngle();
        System.out.printf("Current Power Output: %.2f W\n", currentPowerOutput);
        System.out.printf("Panel Tilt Angle: %.2f°\n", panelTiltAngle);
    }
    private void adjustTiltAngle() {
        if (currentPowerOutput < minOptimalPower) {
            panelTiltAngle += 5.0;
        } else if (currentPowerOutput > maxOptimalPower) {
            panelTiltAngle -= 5.0;
        }
    }
}

public class c66_SmartSolarSystem {
    public static void main(String[] args) {
        SmartSolarPanel solarPanel = new SmartSolarPanel(500.0, 200.0, 300.0, 30.0);

        // testcase-VT:
        // solarPanel.updatePowerOutput(-150.0);
        // solarPanel.updatePowerOutput(100.0);
        // solarPanel.updatePowerOutput(300.0);
        // solarPanel.updatePowerOutput(-50.0);

        // testcase-FT:
        // solarPanel.updatePowerOutput(-4.9E-324);
        // solarPanel.updatePowerOutput(0.0);
        // solarPanel.updatePowerOutput(-1.7976931348623157E308);
        // solarPanel.updatePowerOutput(2.2250738585072014E-308);

        // testcase-Z3:
        // solarPanel.updatePowerOutput(-150.0);
        // solarPanel.updatePowerOutput(100.0);
        // solarPanel.updatePowerOutput(300.0);
        // solarPanel.updatePowerOutput(-50.0);

        // testcase-UVT:
        solarPanel.updatePowerOutput(-150.0);
        solarPanel.updatePowerOutput(100.0);
        solarPanel.updatePowerOutput(300.0);
        solarPanel.updatePowerOutput(-50.0);
    }
}

```

}

- Coverage report

1. Test case number: 4
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.5 %	4	838	842
✓ src	0.5 %	4	838	842
✓ code	0.5 %	4	838	842
✓ c66_SmartSolarSystem.java	100.0 %	4	0	4
> c66_SmartSolarSystem		0	0	0
> SmartSolarPanel	100.0 %	4	0	4

3. Vibration testing instruction coverage: 96.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	95	20,718	20,813
✓ src	0.5 %	95	20,718	20,813
✓ code	0.5 %	95	20,718	20,813
✓ c66_SmartSolarSystem.java	96.9 %	95	3	98
> c66_SmartSolarSystem	87.5 %	21	3	24
> SmartSolarPanel	100.0 %	74	0	74

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.4 %	3	839	842
✓ src	0.4 %	3	839	842
✓ code	0.4 %	3	839	842
✓ c66_SmartSolarSystem.java	75.0 %	3	1	4
> c66_SmartSolarSystem		0	0	0
> SmartSolarPanel	75.0 %	3	1	4

5. Fuzz testing instruction coverage: 90.8%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.4 %	89	20,724	20,813
✓ src	0.4 %	89	20,724	20,813
✓ code	0.4 %	89	20,724	20,813
✓ c66_SmartSolarSystem.java	90.8 %	89	9	98
> c66_SmartSolarSystem	87.5 %	21	3	24
> SmartSolarPanel	91.9 %	68	6	74

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
✓ VibrationTestingTool	0.5 %	4	838	842
✓ src	0.5 %	4	838	842
✓ code	0.5 %	4	838	842
✓ c66_SmartSolarSystem.java	100.0 %	4	0	4
> c66_SmartSolarSystem		0	0	0
> SmartSolarPanel	100.0 %	4	0	4

7. Z3 instruction coverage: 96.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
✓ VibrationTestingTool	0.5 %	95	20,718	20,813
✓ src	0.5 %	95	20,718	20,813
✓ code	0.5 %	95	20,718	20,813
✓ c66_SmartSolarSystem.java	96.9 %	95	3	98
> c66_SmartSolarSystem	87.5 %	21	3	24
> SmartSolarPanel	100.0 %	74	0	74

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c66_SmartSolarSystem.java	100.0 %	4	0	4
> c66_SmartSolarSystem		0	0	0
> SmartSolarPanel	100.0 %	4	0	4

9. Unimproved V-method instruction coverage: 96.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	95	20,718	20,813
▼ src	0.5 %	95	20,718	20,813
▼ code	0.5 %	95	20,718	20,813
▼ c66_SmartSolarSystem.java	96.9 %	95	3	98
> c66_SmartSolarSystem	87.5 %	21	3	24
> SmartSolarPanel	100.0 %	74	0	74